

University of Calgary
Department of Electrical and Computer Engineering
Lab 2

Due Date: Mon Sep 21, 2026, 11:59 PM

Important Notes:

1. Students may complete this lab individually or in pairs. Groups of more than two students are not permitted.
Collaboration with a partner is encouraged because it enables discussion of course concepts, exchange of ideas, and collaborative learning. Pair work also provides experience with pair programming; a practice commonly used in Agile Software Development.
In pair programming, two students collaborate on the same task. One student serves as the driver, responsible for writing and implementing code, while the other acts as the observer or navigator, reviewing code, verifying correctness, and providing feedback on design and logic. Partners should switch roles regularly to ensure active participation and experience in both responsibilities. Students collaborating remotely should use screen sharing or video-conferencing tools to facilitate effective pair programming.
2. If you choose to work with a partner, submit ***only one report on behalf of the group***, and clearly list both partners' names in the report. The submission of two lab reports containing identical content will be regarded as an act of copying and plagiarism.
3. When submitting your source code files (.cpp and .h), ensure that the following information is included in the header comment at the beginning of each file:
 - a. File Name
 - b. Assignment and exercise number
 - d. Your name
 - e. Submission Date:

Here is an example:

```
/*  
 * File Name: lab2exe_A.h  
 * Assignment: Lab 2 Exercise A  
 * Completed By:  
 *   - student 1 name  
 *   - student 2 name (if applicable)  
 * Submission Date: Sept. 18, 2026  
 */
```

4. The lab report must include a cover page containing the information listed below. This cover page helps facilitate efficient grading and grade entry in the D2L system.

Student Name(s)
Course Name: Advanced System Analysis and Software Design
Course Code: ENSF 480
Assignment Number (e.g., Lab 2)
Submission Date (DD/MM/YYYY)

Marking scheme:

Total Marks: 36

- Exercise A: 16 marks
- Exercise B: 20 marks

Objectives:

Understanding the concept of overloading operators in C++

Understanding the concepts using C++ features such friend and static data member

Understanding the concept of hierarchy, another pillar of OOP, which is known as, inheritance.

Exercise A - Overloading Operators in C++ (16 marks)

In this exercise, you will overload several operators, for classes `DictionaryList`.

- **What to Do:**

Download the files `Lab2_exAmain.cpp`, `dictionaryList.h`, `dictionary.cpp`, from D2L.

Now, compile and run the program to make sure it works. The output will be like what you experienced in lab 1.

In the file `Lab2_exBmain`, you will see that by using `#if 0`, call to the function `test_operator_overloading`, and its implementation is commented out. To allow calling function `test_operator_overloading`, change the conditional compilations to `#if 1`, and try to compile and run the program. Now you will see some errors. These errors are because objects of class `DictionaryList` are using C++ operators that are not overloaded.

Your task is to analyze the provided code, determine which operator requires overloading, and implement the necessary changes in `dictionaryList.h` and `dictionaryList.cpp` to ensure that the program behaves as intended.

Since more than one operator are to be overloaded, the best strategy is to work incrementally. It means in function `test_overloading`, include one overloaded function at a time.

- **Instructions to Submit Exercises A:** Submit the following files to D2L:
 - A **PDF lab report** containing:
 - copies of all submitted `.cpp` and `.h` files; and
 - the output generated by your program.
 - A ZIP archive containing all source code files (`.cpp` and `.h`) used in your solution.

Exercise B - Inheritance in C++ (20 marks)

The concepts of aggregation, composition, and inheritance in C++ and Java are similar, but they are completely different in terms of implementation and syntax. Also, another major difference between the two languages is that C++ unlike Java supports multiple inheritance.

- **What to Do:**

In this exercise, you will implement the following classes: `Point`, `Shape`, `Rectangle`, `Square`, and `GraphicsWorld`, according to the specifications provided below.

Class Point:

The `Point` class models a point within a Cartesian coordinate system. It includes three data members: the x-coordinate, the y-coordinate, and a unique identification number (id). The id is assigned automatically, beginning with 1001 for the first `Point` object, 1002 for the second, and incrementing sequentially. Class `Point` should also have at least the following member functions:

- `Display`: displays x, and y coordinates of the point in the following format:
X-coordinate: #####.##
Y-coordinate: #####.##
- A constructor that initializes its data members.
- **Note: You are not supposed to define a default constructor in this class. Automatic calls to the default constructor will hide some of the important aspects of this assignment (marks will be deducted if you define a default constructor for this class).**
- Access functions, getters and setters, as needed.
- Function `counter()` that returns the number of objects of class `Point` at any time.
- Two distance functions that return the distance between two points. One of the two must be a static function.

Implement the `Point` class using two files: `point.h` and `point.cpp`.

Class Shape:

This class is the base class or the ancestor of several classes, including class `Rectangle`, `Square`, `Circle`, etc. It should support basic operations and structures that are common among the children of this class. This class should have an object of class `Point` called `origin`, and a `char` pointer called `shapeName` that points to a dynamically allocated memory space, allocated by the class constructor. This class should also have several functions and a constructor as follows:

- A constructor that initializes its data members.
- **No default constructor**
- A destructor that de-allocates the memory space allocated dynamically for `shapeName`
- `getOrigin`: returns a reference to point `origin`. The reference should not allow the x and y values of point to be modified through this reference.
- `getName`: returns the name of the shape.
- `display`: prints on the screen the shape's name, x and y coordinates of point `origin`, in the following format:
Shape Name:
X-coordinate:
Y-coordinate:
- Two distance functions:
`double distance (Shape& other);`
`static double distance (Shape& the_shape, Shape& other);`

- `move`: changes the position of the shape, the current x and y coordinates, to $x+dx$, and $y+dy$. The function's interface (prototype) should be:

```
void move (double dx, double dy);
```

Implement the Shape class using two files: `shape.h` and `shape.cpp`.

Class Square:

This class is supposed to be derived from class Shape, and should have one data member, called `side_a`, and in addition to its constructor should have a constructor and several member functions as follows:

- One constructor that initializes its data members with its arguments supplied by the user.
- **No default constructor.** Marks will be deducted if a default constructor is defined for this class.
- `area`: returns the area of a square
- `perimeter`: returns the perimeter of a square
- Access functions, getters and setters, as needed.
- `display`: that displays the name, x and y coordinates of the origin, `side_a`, area, and perimeter of a square object in the following format:

```
Square Name:
X-coordinate:
Y-coordinate:
Side a:
Area:
Perimeter;
```
- More Functions, if needed.

Implement the Square class using two files: `square.h` and `square.cpp`.

Class Rectangle:

Class Rectangle that is derived from class Square needs to have one more side called `side_b`. This class, in addition to its constructor (no default constructor), should support the following functions:

- `area`: calculates and returns the area of a rectangle
- `perimeter`: calculates and returns the perimeter of a rectangle
- getters and setters: retrieve or change the values of its private data members.
- `display`: displays the name, and x and y coordinate origin of the shape, `side_a`, `side_b`, area, and perimeter of a rectangle object, in the following format:

```
Rectangle Name:
X-coordinate:
Y-coordinate:
Side a:
Side b:
Area:
Perimeter;
```
- More Functions, if needed.

Implement the Rectangle class using two files: `rectangle.h` and `rectangle.cpp`.

Class GraphicsWorld:

This class should have only a function called `run`, which declares instances of the above-mentioned classes as its local variable. This function should first display a short message about the author(s) of the program and then start testing all the functions of the classes in this system. A sample code segment of function run

is given below. You are recommended to use conditional compilation directives to test your code (one class at a time).

```
void GraphicsWorld::run() {
    #if 0 // Change 0 to 1 to test Point
        Point m (6, 8);
        Point n (6,8);
        n.setx(9);cout << "\nExpected to display the distance between m and n is: 3";
        cout << "\nThe distance between m and n is: " << m.distance(n);
        cout << "\nExpected second version of the distance function also print: 3";
        cout << "\nThe distance between m and n is again: "
            << Point::distance(m, n);
    #endif // end of block to test Point
    #if 0 // Change 0 to 1 to test Square
        cout << "\n\nTesting Functions in class Square:" <<endl;
        Square s(5, 7, 12, "SQUARE - S");
        s.display();
    #endif // end of block to test Square
    #if 0 // Change 0 to 1 to test Rectangle
        cout << "\n\nTesting Functions in class Rectangle:";
        Rectangle a(5, 7, 12, 15, "RECTANGLE A");
        a.display();
        Rectangle b(16, 7, 8, 9, "RECTANGLE B");
        b.display();
        double d = a.distance(b);
        cout << "\nDistance between square a, and b is: " << d << endl;
        Rectangle rec1 = a;
        rec1.display();
        cout << "\n\nTesting assignment operator in class Rectangle:" <<endl;
        Rectangle rec2 (3, 4, 11, 7, "RECTANGLE rec2");
        rec2.display();
        rec2 = a;
        a.set_side_b(200);
        a.set_side_a(100);
        cout << "\n\nExpected to display the following values for objec rec2: " << endl;
        cout << "Rectangle Name: RECTANGLE A\n" << "X-coordinate: 5\n" << "Y-coordinate: 7\n"
            << "Side a: 12\n" << "Side b: 15\n" << "Area: 180\n" << "Perimeter: 54\n" ;
        cout << "\n\nIf it doesn't there is a problem with your assignment operator.\n" << endl;
        rec2.display();
        cout << "\n\nTesting copy constructor in class Rectangle:" <<endl;
        Rectangle rec3 (a);
        rec3.display();
        a.set_side_b(300);
        a.set_side_a(400);
        cout << "\n\nExpected to display the following values for objec rec2: " << endl;
        cout << "Rectangle Name: RECTANGLE A\n" << "X-coordinate: 5\n" << "Y-coordinate: 7\n"
            << "Side a: 100\n" << "Side b: 200\n" << "Area: 20000\n" << "Perimeter: 600\n" ;
        cout << "\n\nIf it doesn't there is a problem with your assignment operator.\n" << endl;
        rec3.display();
    #endif // end of block to test Rectangle
    #if 0 // Change 0 to 1 to test using array of pointer and polymorphism
        cout << "\n\nTesting array of pointers and polymorphism:" <<endl;
        Shape* sh[4];
        sh[0] = &s;
        sh[1] = &b;
        sh[2] = &rec1;
        sh[3] = &rec3;
        sh[0]->display();
        sh[1]->display();
        sh[2]->display();
        sh[3]->display();
    #endif // end of block to test array of pointer and polymorphism
}
```

Implement the GraphicsWorld class using graphicsWorld.h and graphicsWorld.cpp.

- **Instructions to Submit Exercises A:** Submit the following files to D2L:
 - A **PDF lab report** containing:
 - copies of all submitted .cpp and .h files; and
 - the output generated by your program.
 - A ZIP archive containing all source code files (.cpp and .h) used in your solution.

Acknowledgment:

This laboratory exercise was originally developed by M. Moussavi, PhD, PEng. The author's contribution to the development of these lab materials is gratefully acknowledged.